

SESSION 2: FEDERATED LEARNING SYSTEMS

What is Federated Learning

- Decentralized training across many edge devices or clients
- Data stays on device; only model updates or gradients are shared

Why it matters

- Privacy preservation and regulatory compliance
- Lower central bandwidth use; clients compute locally

Core challenges

- Clients are intermittent and heterogeneous (CPU/GPU, battery, network)
- Non-IID data across clients; drift and imbalance
- Faults and stragglers; limited communication rounds

FedAvg (Federated Averaging) protocol

- Round t :
 - Server broadcasts current model to selected clients
 - Clients train locally for a few epochs on their data
 - Clients send model deltas or updated weights to server
 - Server aggregates (weighted average by client data size) $\rightarrow$ new global model

Practical tips

- Client sampling: pick a subset each round to scale and reduce stragglers
- Weighting: average by number of local examples to reduce bias
- Compression: quantize/sparsify updates to save bandwidth
- Security/privacy: secure aggregation, differential privacy, device attestation
- Evaluation: track both global and per-client metrics; handle skewed datasets

Client Scheduling and Participation

Why scheduling matters

- Clients are heterogeneous and intermittent; naïve selection creates bias and slow rounds

Eligibility criteria (examples)

- Sufficient battery and charging state
- Unmetered/high-quality network; device idle and not in active use
- Minimum storage/CPU/GPU/RAM; recent app/version; security/attestation passed

Selection strategies

- Random sampling from eligible pool each round
- Stratified sampling by geography, device class, or data attributes to reduce bias
- Priority/credit systems to ensure fairness and fresh participation
- Cap per-client frequency to avoid overusing fast devices

Availability patterns to consider

- Time zones and diurnal patterns
- Usage behavior (work hours vs. overnight charging)
- Network variability by region/carrier; hardware capability differences

Robustness and performance

- Over-select (invite $K \gg \text{target}$) to tolerate dropouts; accept first N that check in
- Deadlines and partial aggregation; early stopping if enough updates arrive
- Adaptive cohort size: increase when stragglers are common
- Secure eligibility checks; anonymized telemetry for scheduling
- Metrics to monitor
- Round completion time, acceptance rate, dropout rate, coverage/fairness by strata
- Contribution quality/weight (examples seen, loss improvement)

Communication- Efficient Protocols

Core idea: cut bytes per round without hurting accuracy

Techniques

- **Quantization:** send updates in 8/4/2 bits with per-tensor scales
- **Sparsification:** top-k or thresholded updates; error feedback to recover dropped mass
- **Structured updates:** layer-wise or last-layer only; low-rank adapters (LoRA/PEFT)
- **Delta/periodic sync:** send changes since last round; skip small updates

Practicalities

- Clip and add DP noise if required; secure aggregation friendly
- Calibrate k/bit-width; monitor loss drift and add error feedback
- Compress after aggregation when possible to reduce server load

Federated Personalization

Goal: balance one global model with user-specific adaptation

Strategies

- Local fine-tuning: keep a private head or small layers and adapt on-device
- Split models: shared backbone + personal head; only backbone is aggregated
- Meta/regularized training: optimize global loss + personalization term

Practical tips

- Cache small personal params; don't upload them
- Control drift with lightweight regularization and early stopping
- Evaluate both global and per-segment metrics



Non-IID Data Challenges

What's hard

- Client data are skewed and imbalanced; distributions drift over time

Diagnose

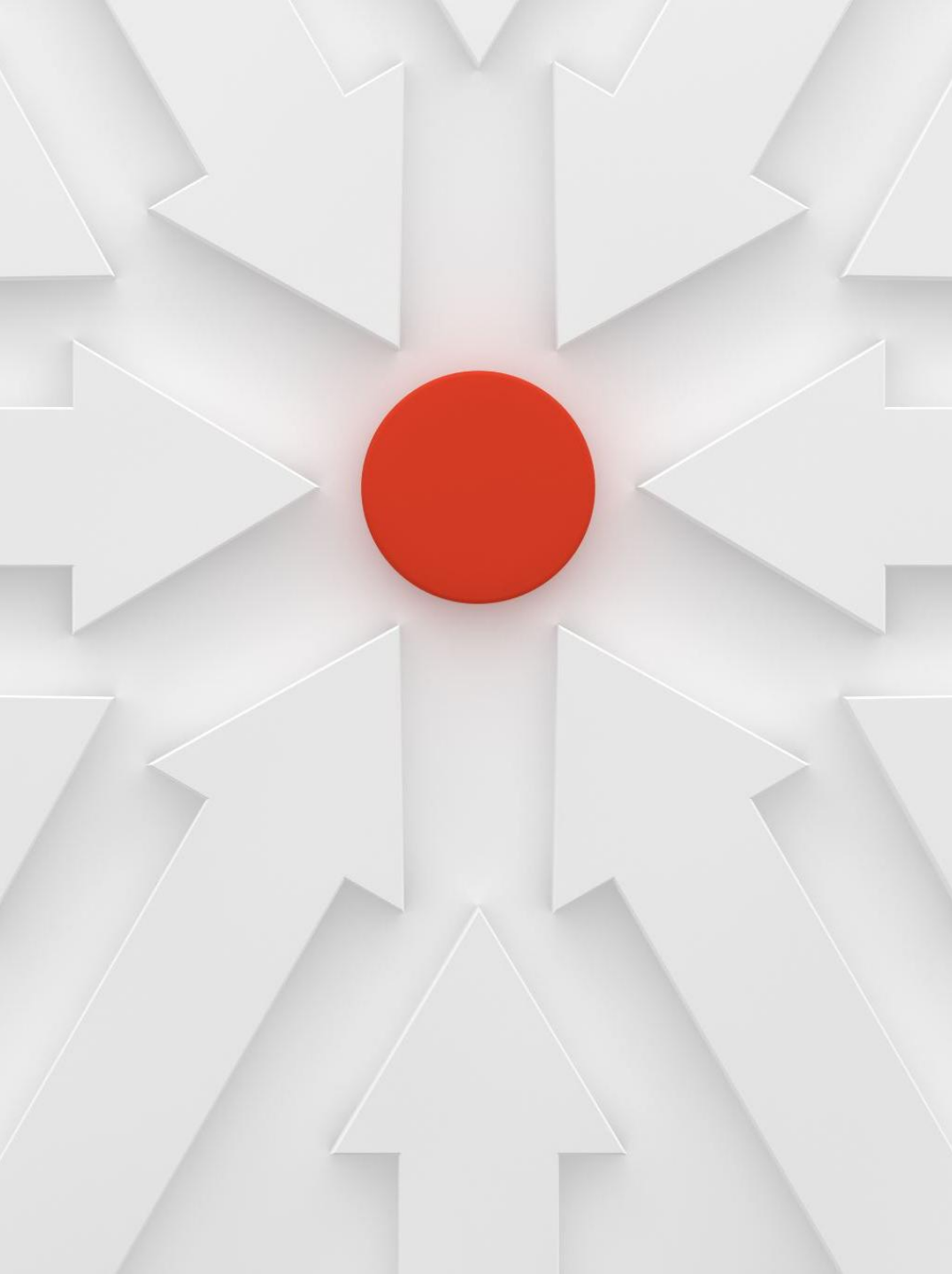
- Compute per-client stats (label/feature histograms, entropy, KS tests)
- Cluster similar clients; track drift across rounds

Adapt aggregation

- Weight by data size, quality, and similarity to global
- Use cohort-wise or personalized aggregation instead of one-size-fits-all

Practical guardrails

- Robustness: clip norms, outlier filtering
- Evaluation: group metrics and worst-case performance



Federated Learning Performance

Convergence factors

- Local steps vs. rounds; data and system heterogeneity; client sampling

Efficiency trade-offs

- More local steps → fewer rounds but risk of drift
- Communication cost vs. accuracy curve

Key metrics

- Rounds to target accuracy; bandwidth per round/total
- Participation rate and straggler impact
- On-device energy/time per round